Master Document 5: End-to-End User Workflows & Data Lifecycle

Project Name: Agri-Grid: Smart Agriculture & Food Security Ecosystem
Project Lead: Ameer Muawiyah (Roll No: 1512)
Context: System Workflows, Database Triggers & API Handshakes

1. Document Purpose

This document maps the exact step-by-step journeys of the system's users and traces the lifecycle of the data as it flows from the frontend UI, through the Python routing engine, to the external APIs, and finally into the SQL database. It serves as the master logic blueprint for the Core Backend Engineer and the Database Administrator.

2. Workflow 1: The Commercial Supply Chain Loop

(Actors: Farmer ➔ B2B Buyer ➔ Transporter)

Step 1: Yield Prediction & Listing Publishing (Farmer)

- **User Action:** The Farmer inputs a crop type (e.g., Wheat) and planting date, then clicks "Forecast & Publish."
- **Data Lifecycle:**

1. The frontend sends a JSON payload with the farm's GPS coordinates to the Python backend.
2. The backend triggers a request to external Weather/Soil APIs (e.g., OpenWeatherMap, SoilGrids) to fetch 14-day telemetry and historical data.
3. The Data Architect's algorithm processes this data to estimate harvest date and yield volume.
4. The DBA's system generates a unique Batch_ID and executes an SQL INSERT into the Listings table, setting the status to "Active."

Step 2: Escrow Checkout & Dispatch Creation (B2B Buyer)

- **User Action:** The Buyer filters the catalog, selects the Farmer's listing, sets Cold-Chain parameters (e.g., < 6°C), and submits payment.
- **Data Lifecycle:**

1. The frontend triggers a complex SQL SELECT with JOIN statements across Farmers and Listings to render the catalog.
2. Upon checkout, the backend executes an ACID-compliant transaction: it updates the Listings status to "Sold", inserts the financial value into the Escrow_Holdings table, and automatically generates a new row in the Deliveries table (The Dispatch Manifest).

Step 3: The Custody Handshake (Transporter)

- **User Action:** The Transporter arrives at the farm and scans the QR code on the physical cargo using their mobile browser.
- **Data Lifecycle:**

1. The QR scan sends a timestamped webhook to the backend.

2. An SQL UPDATE changes the Deliveries table status from "Awaiting Pickup" to "In Transit."
3. The backend pushes an automated alert to the B2B Buyer's dashboard indicating the cargo is on the move.

Step 4: Transit Telemetry Logging (Transporter)

- **User Action:** The Transporter drives through rural areas. The mobile app continuously logs the truck's internal temperature.
- **Data Lifecycle:**

1. If the device loses 4G/5G, the frontend utilizes HTML5 Local Storage (Flash Caching) to temporarily store the JSON temperature payloads.
2. Upon network reconnection, the frontend bulk-syncs the data to the backend, which executes batch INSERT operations into the Telemetry_Logs table, linking them to the specific Batch_ID.

Step 5: Quality Acceptance & Escrow Release (Buyer & Transporter)

- **User Action:** The Transporter delivers the food. The Buyer signs the digital Proof of Delivery (PoD) pad and clicks "Accept Delivery."
- **Data Lifecycle:**

1. The digital signature is converted into a binary blob and stored securely.
2. The backend executes the final financial transaction: it deletes/flags the row in Escrow_Holdings and UPDATEs the Farmer_Wallet table with the final payment amount. The Batch_ID lifecycle is marked as "Completed."

3. Workflow 2: The Telemedicine Diagnostic Loop

(Actors: Farmer ➔ Agronomist)

Step 1: Diagnostic Ticket Creation (Farmer)

- **User Action:** The Farmer uploads a photo of a diseased leaf and selects symptoms.
- **Data Lifecycle:** The image file is sent to a secure cloud storage bucket. The backend executes an INSERT into the Diagnostic_Tickets table with the image URL, farm coordinates, and a "Pending" status.

Step 2: Expert Triage & Contextual API Fetching (Agronomist)

- **User Action:** The Agronomist logs in, filters the queue, and clicks "Claim Ticket" to investigate.
- **Data Lifecycle:**

1. An SQL UPDATE assigns the Agronomist's User_ID to that specific ticket, preventing duplicate work.
2. *Crucial Step:* The backend automatically pings the external Weather APIs using the farm's coordinates, fetching the last 14 days of rainfall and humidity to display alongside the photo, providing environmental context for the disease.

Step 3: Prescription Dispatch (Agronomist)

- **User Action:** The Agronomist submits a digital treatment plan.
- **Data Lifecycle:** The backend updates the ticket status to "Resolved" and pushes the JSON prescription payload to the Farmer. (UI/UX developer may utilize WebSockets to

trigger an instant screen notification for the Farmer).

4. Workflow 3: The Sustainability & Audit Loop

(Actors: Farmer ➔ Compliance Officer)

Step 1: Subsidy Application (Farmer)

- **User Action:** The Farmer uploads proof of a sustainable farming practice (e.g., reduced water usage).
- **Data Lifecycle:** Data is pushed to the Subsidy_Claims table with a "Pending Review" status.

Step 2: API Verification Engine (Compliance Officer)

- **User Action:** The Officer opens the claim to review it against real-world data.
- **Data Lifecycle:** The Python backend runs a comparative algorithm. It queries the Subsidy_Claims data and compares it directly against historical API telemetry (e.g., verifying if the weather actually dictates the soil moisture levels the farmer claims). The UI renders a "Confidence Score" (e.g., 88% Match).

Step 3: Grant Disbursement (Compliance Officer)

- **User Action:** The Officer clicks "Approve & Disburse."
- **Data Lifecycle:** An ACID-compliant SQL transaction deducts the grant amount from the Admin_Budget table and adds it to the Farmer_Wallet table. A read-only event is permanently logged in the Audit_Logs table for government transparency.

5. Workflow 4: The Public Provenance Loop

(Actors: Public Consumer)

Step 1: QR Scan & Timeline Generation

- **User Action:** A grocery store consumer scans the final product's QR code with their phone camera. No login is required.
- **Data Lifecycle:**

1. The backend receives the Batch_ID from the QR URL.
2. The backend executes a massive, read-only SQL JOIN query, pulling the origin location from the Farmers table, the health history from the Diagnostic_Tickets table, and the cold-chain temperature limits from the Telemetry_Logs table.
3. The frontend renders this aggregated JSON payload as a beautiful, vertical "Farm-to-Table" visual timeline.